Launch27 Integration Guide

1. What This Integration Does

The Newo AI Agent integrates with Launch27 so your Agent can take real cleaning and local-service bookings end-to-end inside a conversation, with no manager callback or external scheduling link.

Once connected, the AI Agent can:

- Check Launch27 availability for a given date and service in real time
- Create new bookings directly in Launch27 (single or recurring frequency)
- Cancel an existing Launch27 booking, including handling late-cancel and recurring confirmations
- Recognize returning callers by reading their stored Launch27 bookings

In plain terms: your callers can ask "do you have anything Tuesday morning?" or "cancel my Friday cleaning" and the Agent handles it the same way a trained dispatcher would — directly against your Launch27 tenant.

2. Before You Start

Make sure you have:

- an active Launch27 tenant (the subdomain you log in with, for example `acme` for `https://acme.launch27.com`)
- a Launch27 customer login (email + password) that can authenticate via the public `/login` endpoint
- access to the Newo project builder for your project
- permission to edit project settings and publish

2.1 Find your Launch27 tenant domain

Log in to Launch27 in a browser and look at the URL. The subdomain — the part between `https://` and `.launch27.com` — is your **Tenant Domain**. For `https://acme.launch27.com`, the value you'll paste into Newo is just `acme` (no protocol, no trailing slash).

Figure S01. The Launch27 URL bar — copy only the subdomain (`acme`), not the full URL.

2.2 Identify the Launch27 customer login account

Pick the Launch27 user whose email and password Newo will use to log in via the public `/login` endpoint. This must be a customer-login-capable account on your tenant. Open Launch27's settings/profile page for that user so you have the exact email at hand.

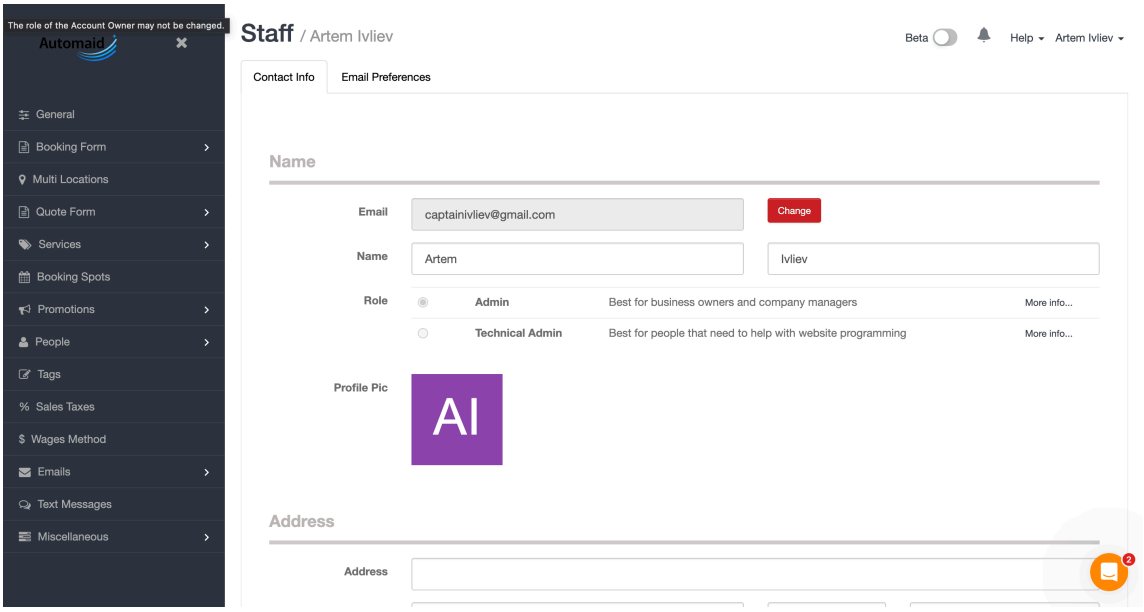

Figure S02. The Launch27 account whose email + password will be used for publish-time login.

2.3 Choose your sign-in mode

You will use one of the following:

- **Email + password (Recommended)** — Newo logs in to Launch27 at publish time and stores the returned JWT bearer token automatically.
- **Email + password + 2FA token** — for tenants that require an authenticator code on `/login`. You paste the current 6-digit code into the **Customer Login OTP Token** attribute right before publishing.

> ***Which should I use?*** *Use plain email + password unless your Launch27 tenant has 2FA enforced. If it does, paste a fresh OTP code immediately before clicking* ***Publish All*** *— the code expires quickly and is only consumed once during publish-time login.*

3. Connecting for the First Time

3.1 Connect with Email + Password (Recommended)

1. Open your project in Newo and go to **Builder → Attributes**.
2. Find the **Module - Launch27Integration** group.
3. Fill in the connection fields:
   - **Tenant Domain** — your subdomain only, no protocol. Example: `acme` (for `https://acme.launch27.com`).
   - **Customer Login Email** — the Launch27 account used for the public API.
   - **Customer Login Password** — the password for that account.
   - Leave **Bearer Access Token** and **Bearer Token Expires At** blank — the integration fills them in.
4. Click **Save**.

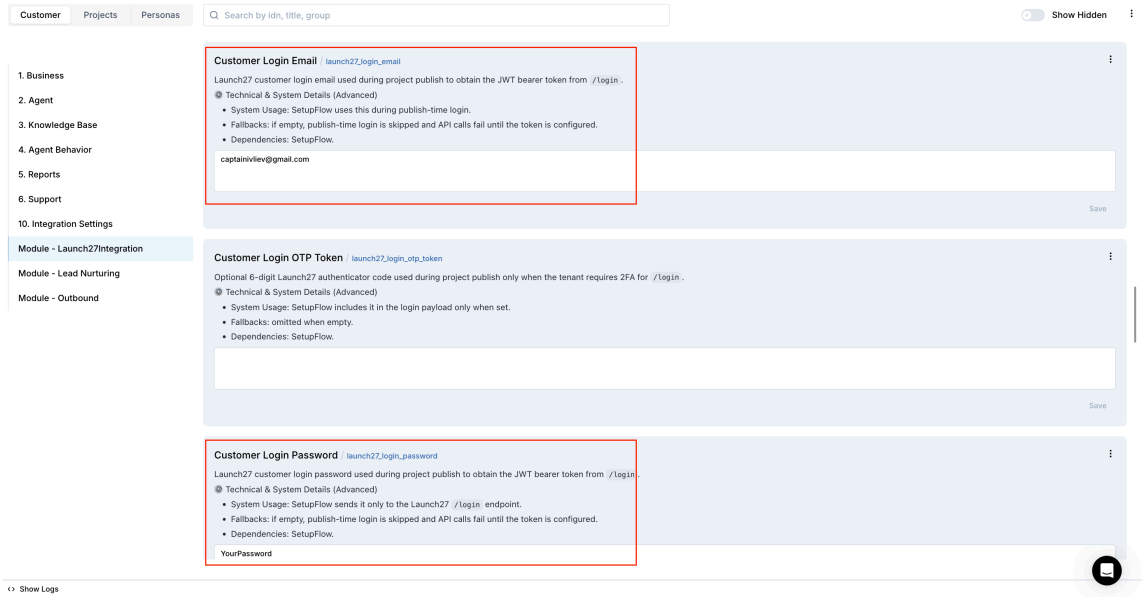

*Figure S03. The Launch27 attributes group with **Tenant Domain**, **Customer Login Email**, and **Customer Login Password** filled. **Bearer Access Token** is still empty — it will be populated by Publish All.*

5. Click **Publish All** at the project level.

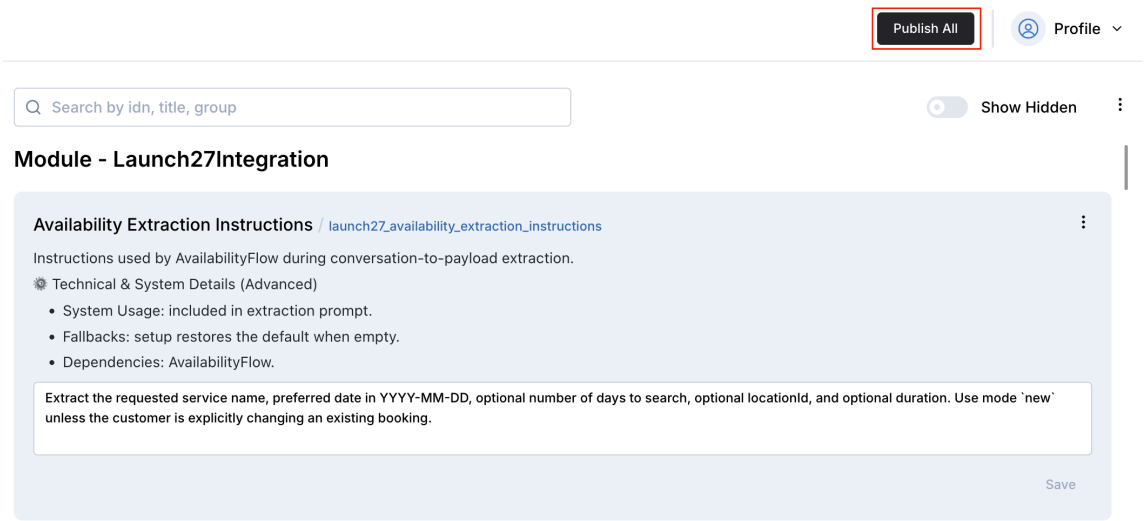

Figure S04. The Publish screen after a successful Publish All — no errors reported.

What happens after Publish:

- Newo logs in to `https://{your-domain}.launch27.com/latest/login` and stores the returned JWT in **Bearer Access Token**.
- The Launch27 services cache is refreshed on the next conversation start (24-hour TTL by default).
- The three Launch27 booking tools (`launch27_check_availability` , `launch27_book_appointment` , `launch27_cancel_appointment`) are registered with the agent, and the generic NAF availability/booking/cancellation tools are explicitly disabled so they cannot fire.

- Launch27-specific scenarios, procedures and intent types are inserted into the agent canvas (unless **Setup Scenarios** is set to `False`).

3.2 Connect with Email + Password + 2FA

Use this method only if your Launch27 tenant requires an authenticator code on `/login`. Confirm 2FA is enforced for your customer-login account in Launch27's security settings before proceeding.

Figure S05. The Launch27 2FA / security settings for the customer-login account, with the authenticator app showing the current 6-digit code.

1. Open your authenticator app and copy the current 6-digit code for your Launch27 account.
2. In Newo (**Builder → Attributes → Module - Launch27Integration**), paste the code into **Customer Login OTP Token**.
3. Make sure **Tenant Domain**, **Customer Login Email**, and **Customer Login Password** are also set.
4. Click **Save** and immediately **Publish All** — the OTP only stays valid for ~30 seconds.

> ***Important:*** *the OTP is only used during publish-time login. After login succeeds, the JWT in* ***Bearer Access Token*** *is what authenticates every subsequent Launch27 call. You can clear the OTP field after publish.*

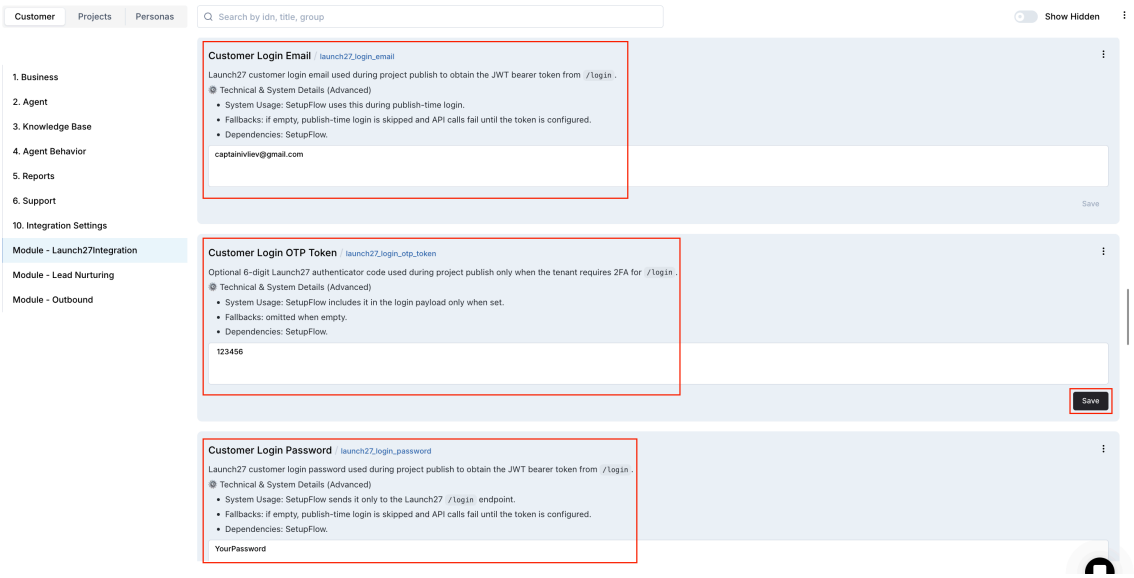

Figure S06. The Newo attributes screen with ***Customer Login OTP Token*** *filled in addition to the email/password from Figure S03.*

4. Settings Reference

You can find all settings in **Builder → Attributes → Module - Launch27Integration**. After a successful publish, the connection settings you filled in are visible, the bearer token is populated, and the services cache is updated on the next conversation start.

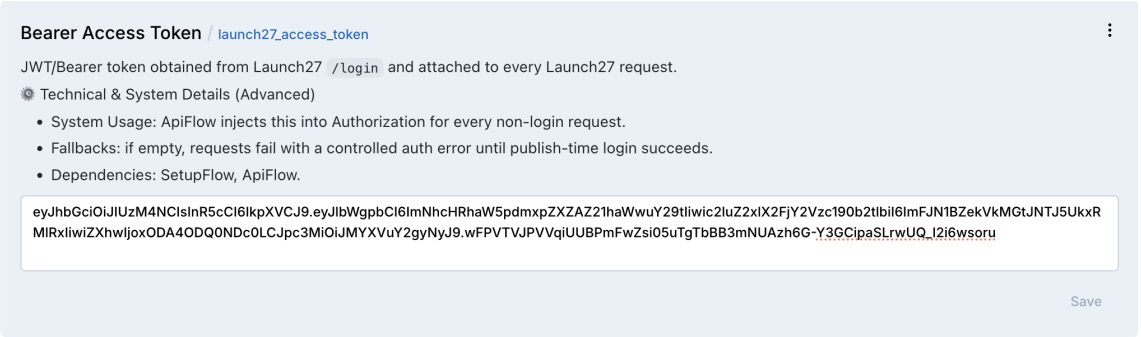

*Figure S07. The full Launch27 attributes group after a successful publish — **Bearer Access Token** is populated and **Services Cache Updated At** has a timestamp.*

4.1 Connection Settings

Setting	What it does	Required
Tenant Domain (`launch27_domain`)	Launch27 subdomain (e.g. `acme`) — combined with the URL template to build every API call.	Yes
Base URL Template (`launch27_base_url_template`)	URL template with `{domain}` placeholder. Default: `https://{domain}.launch27.com/latest`.	Yes (default provided)
Customer Login Email (`launch27_login_email`)	Email used for publish-time login to obtain the JWT.	Yes
Customer Login Password (`launch27_login_password`)	Password for that login.	Yes
Customer Login OTP Token (`launch27_login_otp_token`)	6-digit authenticator code, used only when the tenant requires 2FA on `/login`.	Only if 2FA is enforced
Bearer Access Token (`launch27_access_token`)	JWT injected into the `Authorization` header of every non-login call. Auto-populated by publish-time login.	Auto
Bearer Token Expires At (`launch27_access_token_expires_at`)	Optional ISO timestamp of token expiry (reserved for future authenticated customer flows).	No

4.2 Feature On/Off Switches

Each capability can be independently enabled or disabled. When a switch is `False`, the corresponding flow exits silently before any extraction or API call.

Setting	Default	What it controls

Availability Feature Enabled (`launch27_feature_availability_enabled`)	True	Whether `launch27_check_availability` runs
Booking Feature Enabled (`launch27_feature_booking_enabled`)	True	Whether `launch27_book_appointment` runs
Cancellation Feature Enabled (`launch27_feature_cancellation_enabled`)	True	Whether `launch27_cancel_appointment` runs
Setup Scenarios (`launch27_setup_scenarios`)	True	Whether CanvasFlow installs Launch27 scenarios, procedures, and intent types when the canvas is built

> ***Recommended defaults for a new setup:*** *keep all four* `True`*. Disable an individual feature only when you want the Agent to refuse a capability while keeping the others.*

4.3 Booking Defaults

Setting	What it does
Default Frequency ID (`launch27_default_frequency_id`)	Launch27 frequency ID used when the conversation does not specify one. **Required** for booking — booking errors out if both extracted and default values are empty.
Default Arrival Window (minutes) (`launch27_default_arrival_window`)	Used when the conversation does not specify an arrival window. Default: `60`.

Where to find your Frequency ID: open Launch27's frequency settings (one-time, weekly, biweekly, etc.). The ID is visible in the URL of the frequency edit page (`https://{yourdomain}.launch27.com/admin/frequencies/1/edit` → frequency ID is `1`).

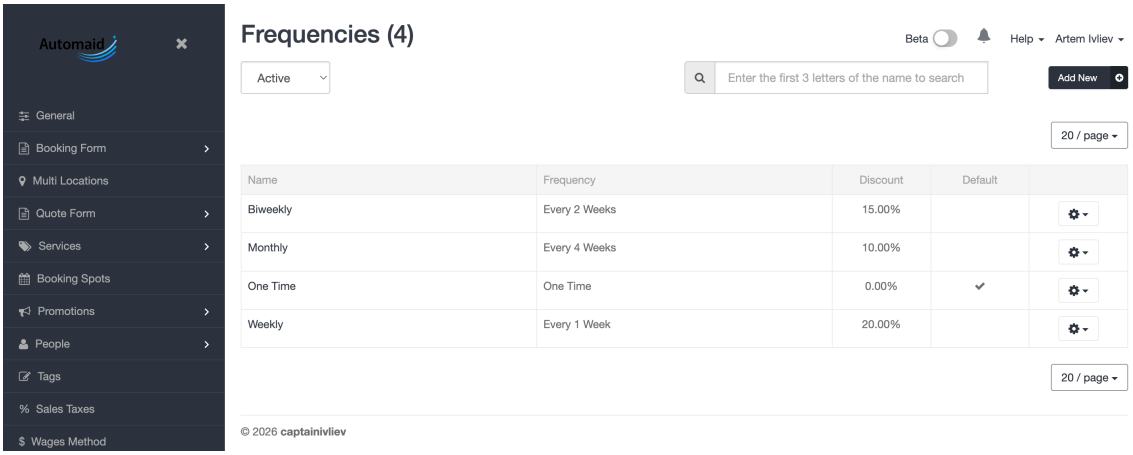

Figure S08. The Launch27 frequency settings — copy the ID of the cadence you want as your default.

4.4 Services Cache

The integration keeps a local cache of your Launch27 services to match the caller's request to a real bookable service.

Setting	What it does
Services Cache (`launch27_services_cache`)	Cached `/booking/services` response. Source of truth for service matching in availability and booking. Auto-populated.
Services Cache Updated At (`launch27_services_cache_updated_at`)	Timestamp of the last successful cache refresh. Empty means "stale, refresh on next conversation".
Services Cache TTL (seconds) (`launch27_services_cache_range_seconds`)	How long the cache stays fresh. Default: `86400` (24h).
Services Cache Tags (`launch27_services_cache_tags`)	Cached `tags_info` metadata for future filtering logic.

4.5 LLM Extraction Schemas and Instructions

These control how the agent extracts structured payloads from conversation history before calling Launch27. You should rarely edit these — Setup restores defaults when blanks are detected.

Setting	Used by
Availability Extraction Schema / Instructions	AvailabilityFlow when running `launch27_check_availability`
Booking Extraction Schema / Instructions	BookingFlow when running `launch27_book_appointment`
Cancellation Extraction Schema / Instructions	CancellationFlow when running `launch27_cancel_appointment`

5. What the AI Agent Can Do

The Launch27 integration exposes three custom tools and disables the generic NAF availability/booking/cancellation tools so they cannot interfere.

5.1 Check Availability

The Agent calls `POST /booking/spots` against your Launch27 tenant for a given date, service, and (optionally) location, then reads the returned slots back to the caller.

Example: *"Can you do a two-bedroom deep clean Thursday afternoon?"* → the agent extracts the service name (`Two Bedroom Home`), preference (`deep cleaning`), and date, matches the service against the local services cache, and queries `/booking/spots` for that day. Real returned slots are spoken to the caller.

How it works automatically:

- **Service matching** — the requested service is matched (case-insensitive substring) against the Launch27 services cache. If no match and no explicit duration is given, the agent asks the caller to confirm the exact service or home-size.
- **Date normalization** — past dates without a year are rolled to the next future occurrence.
- **In-progress guard** — duplicate availability requests inside the same session are skipped.

You will see in Launch27: no record at this step — availability is read-only.

5.2 Book an Appointment

The Agent books the appointment the caller confirms via `POST /booking`.

What happens automatically:

1. The agent extracts `serviceName`, `customerInfo` (first name, last name, email, phone, optional company), `address`, `city`, `state`, `zip`, `bookingDateTime`, optional `arrivalWindow`, optional `frequencyId`, optional `locationId`, `customerNotes` from the conversation.
2. Customer info is filled in from the persona's stored `full_name`, `email`, and `provided_phone_number_with_country_code` when missing.
3. The service is matched against the services cache; the agent refuses to book if it cannot confidently match.
4. Pricing parameters and extras from the matched service are included in the request.
5. On success, the booking is appended to the persona's `bookings` attribute (with `booking_id`, `datetime`, and `service_name`) so future sessions can recognize it.

You will see in Launch27: a new booking in the customer's record. Payment method is sent as `cash` by default.

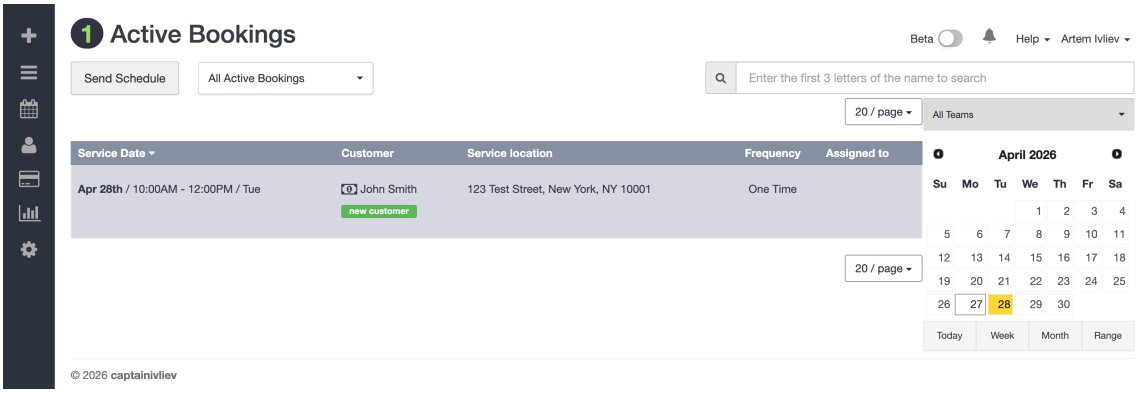

Figure S09. The new booking as it appears in the Launch27 dashboard — customer name, service, and date/time are populated.

5.3 Cancel an Appointment

The Agent cancels an existing booking via `POST /customer/bookings/{booking_id}/cancel`.

How it works automatically:

- The booking ID is taken **only from the caller's saved bookings** (the `bookings` persona attribute) — the agent will not invent IDs.
- A cancellation reason, late-cancel confirmation, fee acknowledgment, or recurring confirmation are included in the body **only when the caller explicitly stated them**.
- On success, the booking is removed from the persona's `bookings` list so it cannot be re-cancelled.

You will see in Launch27: the booking moves to a cancelled state in your dashboard. If Launch27 returns a late-cancel fee or recurring confirmation prompt, the agent surfaces it to the caller and waits for explicit confirmation before retrying.

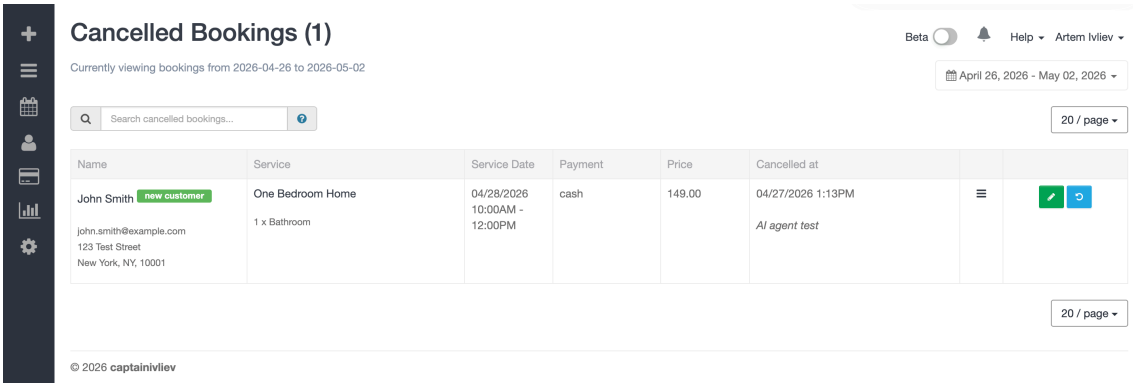

Figure S10. The same booking from Figure S09 after the agent cancelled it — Launch27 shows it in the Cancelled state.

5.4 Recognize Returning Clients

When the caller has previously booked through the agent, the booking is in the persona's `bookings` attribute (`{ datetime, meta: { booking_id, service_name } }`). The agent uses that list to answer questions like *"when is my next cleaning?"* and to scope cancellation requests.

You will see in Launch27: no record at this step — recognition uses Newo's persona store, not Launch27.

6. How to Test That Everything Works

Before going live, run through this checklist on a demo contact.

6.1 After Setup

After clicking **Publish All**, confirm:

- ☐ **Bearer Access Token** is no longer empty (publish-time login succeeded).
- ☐ **Module - Launch27Integration** group shows your domain, email, and password.
- ☐ No errors appear in the publish status.
- ☐ On the first conversation start, **Services Cache** populates with your real Launch27 services.

6.2 Test Availability

1. Start a test chat or call session.
2. Ask: *"Do you have anything next Tuesday morning for a one-bedroom cleaning?"*
3. The agent should match `One Bedroom Home` (or your equivalent) against the services cache and read back real Launch27 slots.

6.3 Test Booking

1. Continue from the availability test.
2. Confirm a slot and provide a test name, email, phone, and full address.
3. Check Launch27 — a new booking should appear with the chosen service, date/time, and customer info.
4. Confirm the booking is also visible in the persona's `bookings` attribute in Newo.

6.4 Test Cancellation

1. From the same session (or a new session for the same caller), ask: *"Cancel my Tuesday cleaning."*
2. The agent must use the saved `booking_id` from `bookings` — it should not ask the caller for the ID.
3. Check Launch27 — the booking should be cancelled.
4. Confirm the booking is removed from the persona's `bookings` attribute.

6.5 Test Returning Client Recognition

1. Wait for the booking from 6.3 to be saved.
2. Start a new session as the same caller (same phone or persona).
3. Ask: *"When is my next cleaning?"* — the agent should reference the saved booking instead of asking from scratch.

7. Reconnecting and Rotating Credentials

When do I need to reconnect?

- Your Launch27 password was rotated.
- The bearer token was revoked or expired.
- Your tenant turned 2FA on for `/login`.
- You renamed your Launch27 subdomain.

How to reconnect

1. **Builder → Attributes → Module - Launch27Integration**.
2. Update **Customer Login Email**, **Customer Login Password**, and (if 2FA is now required) **Customer Login OTP Token**.
3. Optionally clear **Bearer Access Token** and **Bearer Token Expires At** — they will be repopulated by publish-time login.
4. Click **Save**.
5. Click **Publish All**.

The safe reconnect rule

Any time you change credentials, always do all three:

1. **Save** the attributes
2. **Publish All**
3. Run at least one live test (a `launch27_check_availability` call is enough to confirm the bearer token is valid)

8. Troubleshooting

"Launch27 access token is required but is not configured"

Likely cause: publish-time login failed (wrong email/password, missing OTP, or wrong domain) and the **Bearer Access Token** is still empty.

Fix:

1. Verify **Tenant Domain** has no protocol and no trailing slash (e.g. `acme`, not `https://acme.launch27.com/`).
2. Verify the email/password log in successfully when used directly on `https://{domain}.launch27.com`.

3. If the tenant requires 2FA, paste a fresh **Customer Login OTP Token** and **Publish All** within 30 seconds.

"I need the exact Launch27 service type before I can check availability"

Likely cause: the caller's requested service did not match any entry in the services cache (case-insensitive substring match against `name`).

Fix:

- Ask the caller for the exact service or home-size option (e.g. *One Bedroom Home*).
- Confirm **Services Cache** is populated by checking the attribute value — if it's empty, the cache refresh has not run yet. Start a new conversation to trigger `UpdateCacheSkill` on `conversation_started`.

"I could not confidently match the requested service to a Launch27 service"

Likely cause: the booking step received a service name not present in the services cache.

Fix:

- Confirm the cache contains the expected service. The cache refreshes when **Services Cache TTL (seconds)** is exceeded since **Services Cache Updated At**.
- Force a refresh by clearing **Services Cache Updated At** and starting a new conversation.

"A Launch27 frequency ID is required"

Likely cause: Default Frequency ID is empty and the caller did not specify one.

Fix:

1. In Launch27, find the frequency ID for your default cadence (e.g. one-time).
2. Set it as **Default Frequency ID** in the Newo attributes.
3. **Save** and **Publish All**.

"I need the customer's first name, last name, and email before I can book"

Likely cause: the customer info was not gathered in conversation and the persona has no stored `full_name` or `email`.

Fix:

- Continue the conversation: the agent will ask the caller for the missing fields and retry.
- For repeat callers, ensure the persona's `full_name`, `email`, and `provided_phone_number_with_country_code` are populated by upstream identity flows.

"I need the Launch27 booking ID before I can cancel"

Likely cause: the persona's `bookings` attribute is empty for this caller.

Fix:

- Confirm the booking was created through the same persona — bookings are scoped to `GetUser().id`.

- If the booking was created outside Newo, the agent cannot cancel it (booking IDs are taken only from `bookings`, never extracted from free text).

9. FAQ

Does the agent ever fall back to a "manager will call you back" message instead of booking? No. The custom tool descriptions explicitly forbid deferring to a manager callback, SMS follow-up, or self-service booking link. The agent either books, asks for missing info, or surfaces a clear error.

Why are the standard Newo availability / booking / cancel tools listed as disabled? Launch27 ships its own custom tools (`launch27_check_availability` , `launch27_book_appointment` , `launch27_cancel_appointment`) registered via `project_attributes_settings_newo_tools` . The generic NAF tools are explicitly disabled with an always-false condition so the LLM can never call them. The standard `search_booking_tool` is not disabled — the agent still uses it to recognize returning clients before cancellation.

How long is the bearer token valid? That is controlled by Launch27 itself. The integration does not auto-refresh — when the token expires, calls fail with an auth error and you must reconnect (see section 7). **Bearer Token Expires At** is reserved for future authenticated customer flows.

Where are bookings stored? In two places: in your Launch27 tenant (the source of truth) and in the caller's persona under the `bookings` attribute as `[{ datetime, meta: { booking_id, service_name } }]` . The persona copy is what the agent reads to recognize returning callers and scope cancellations.

Can the agent book recurring services? Yes — pass a recurring frequency ID via **Default Frequency ID** or extracted from the conversation. Launch27 may then ask for `confirmed_recurring` on cancellation, which the agent surfaces to the caller before retrying.

What HTTP endpoints does the integration call?

- `POST /login` (publish-time only, to obtain the JWT)
- `GET /booking/services` (services cache refresh)
- `POST /booking/spots` (availability)
- `POST /booking` (create booking)
- `POST /customer/bookings/{booking_id}/cancel` (cancellation)

All endpoints are called against `https://{your-domain}.launch27.com/latest` (configurable via **Base URL Template**).

What happens to in-progress flags between sessions? Session-scoped flags (`launch27_availability_in_progress` , `launch27_booking_in_progress` , `launch27_cancellation_in_progress`) are cleared on `end_session` by each flow's `ClearDataSkill` . The `bookings` list and the matched-service cache for the persona are intentionally preserved across sessions.